NAME: GUNTI RAHUL

ROLL NUMBER: CH.SC.U4CSE24211

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = pd.read_csv('01-linear-regression-dataset-task-1.csv')
```

```
data.head()
```

	unique_key	taxi_id	trip_sta
0	b99416692f5bc01cdb444cbcec7e1105d30d455d	8dd990653cc2793d96b80ae45e928afc9a590b21b491ee...	5/2
1	b99887012c8beccc1dae3f6fcde2fc94ff0ef7b1	d156d4adff01addd62b65753ce67dc8e04bdaf60f7eb14...	5/2
2	b98b6b7afe1f8065d8ba71cd023d9a13f2460f84	beefd3462e3f5a8e854942a2796876f6db73ebbd25b435...	
3	b9865af3673441b1a79973ad08bd9909bd422af4	ef249ea2c31ccb20ce9eb258d3116254a6f9fe05c7cc84...	5
4	e04dbd9a8ba723cc785c4b0398106b72e16d7870	45002471b85d61e62cf602a15145d2bff912e080f50470...	5

5 rows × 23 columns

```
data.shape
```

```
(500, 23)
```

```
data = data.dropna(subset=['trip_seconds', 'trip_miles', 'fare'])
```

```
data['Payment_Card'] = data['payment_type'].map(lambda p: 1 if p == 'Credit Card' else 0)
```

```
data.head()
```

	unique_key	taxi_id	trip_sta
0	b99416692f5bc01cdb444cbcec7e1105d30d455d	8dd990653cc2793d96b80ae45e928afc9a590b21b491ee...	5/2
1	b99887012c8beccc1dae3f6fcde2fc94ff0ef7b1	d156d4adff01addd62b65753ce67dc8e04bdaf60f7eb14...	5/2
2	b98b6b7afe1f8065d8ba71cd023d9a13f2460f84	beefd3462e3f5a8e854942a2796876f6db73ebbd25b435...	5
3	b9865af3673441b1a79973ad08bd9909bd422af4	ef249ea2c31ccb20ce9eb258d3116254a6f9fe05c7cc84...	5/
5	b997997e971c079dd0986102def2de49e879efe5	fa0a84435b292833f8429400e94600f4b71dd17cafbaf5...	5/2

5 rows × 24 columns

```
X = data[['trip_miles', 'trip_seconds', 'Payment_Card']]
y = data['fare']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

▾ LinearRegression ⓘ ?

▸ Parameters

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Fares")
print("----------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual Fare': y_test.values,
    'Predicted Fare': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Fares
----------------------------------------------------------
    Actual Fare  Predicted Fare
0         13.00       11.054762
1          9.25       11.054762
2          4.50        8.538511
3         31.00       34.109787
4         32.00       11.054762
..          ...             ...
92        31.75       30.783105
93         8.00       14.783486
94        98.00       11.109576
95         7.00       13.544643
96        14.00       14.995270

[97 rows x 2 columns]
```

```
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Regression Evaluation Metrics")
```

```
print("-------------------------------------------------- ")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE):  {mse:.2f}")
print(f"R-squared Score (R2):      {r2:.4f}")
```

```
Regression Evaluation Metrics
----------------------------
Mean Absolute Error (MAE):  5.30
Mean Squared Error (MSE):   109.20
R-squared Score (R2):       0.6130
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
data = pd.read_csv('01-linear-regression-dataset-task-2.csv')
```

```
data.head()
```

	Weight	Height	Age	Gender
0	79	1.80	35	Male
1	69	1.68	39	Male
2	73	1.82	25	Male
3	95	1.70	60	Male
4	82	1.87	27	Male

```
data.shape
```

```
(10, 4)
```

```
data['Gender_Male'] = data['Gender'].map({'Male': 1, 'Female': 0})
```

```
data.head()
```

	Weight	Height	Age	Gender	Gender_Male
0	79	1.80	35	Male	1
1	69	1.68	39	Male	1
2	73	1.82	25	Male	1
3	95	1.70	60	Male	1
4	82	1.87	27	Male	1

```
X = data[['Height', 'Age', 'Gender_Male']]
y = data['Weight']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

▾ LinearRegression ⓘ ?

▸ Parameters

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Weights")
print("-------------------------------------------------")
```

```
comparison_df = pd.DataFrame({
    'Actual Wt': y_test.values,
    'Predicted Wt': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Weights
----------------------------
   Actual Wt   Predicted Wt
0          64      56.856510
1          69      79.105761
```

```
mae = mean_absolute_error(y_test, predictions)
mse = mean_squared_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Regression Evaluation Metrics")
print("----------------------------------------------------")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"R-squared Score (R2):      {r2:.4f}")
```

```
Regression Evaluation Metrics
----------------------------
Mean Absolute Error (MAE): 8.62
Mean Squared Error (MSE):   76.58
R-squared Score (R2):      -11.2525
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
data = pd.read_csv('02-logistic-regression-dataset-task-1.csv')
```

```
data.head()
```

	admit	gre	gpa	rank
0	0.0	380.0	3.61	3.0
1	1.0	660.0	3.67	3.0
2	1.0	800.0	4.00	1.0
3	1.0	640.0	3.19	4.0
4	0.0	520.0	2.93	4.0

```
data.shape
```

```
(400, 4)
```

```
X = data[['gre', 'gpa', 'rank']]
y = data['admit']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
▾ LogisticRegression ⓘ ?
▸ Parameters
```

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Admit")
print("----------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual Admit': y_test.values,
    'Predicted Admit': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Admit
----------------------------
    Actual Admit  Predicted Admit
0              0.0               0.0
1              0.0               0.0
2              1.0               0.0
3              0.0               0.0
4              0.0               0.0
..             ...               ...
75             0.0               0.0
76             0.0               0.0
77             0.0               1.0
```

```
78          0.0          0.0
79          0.0          0.0

[80 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print("----------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
----------------------------
Accuracy: 0.6625
Confusion Matrix:
[[51  2]
 [25  2]]
Classification Report:
              precision    recall  f1-score   support

         0.0       0.67      0.96      0.79        53
         1.0       0.50      0.07      0.13        27

    accuracy                           0.66        80
   macro avg       0.59      0.52      0.46        80
weighted avg       0.61      0.66      0.57        80
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
data = pd.read_csv('02-logistic-regression-dataset-task-2.csv')
```

```
data.head()
```

	Disease	Age	Gender	Smoker_status
0	diseased	43	Male	Smoker
1	not diseased	18	Male	Smoker
2	diseased	22	Female	Non-smoker
3	diseased	25	Male	Non-smoker
4	not diseased	45	Female	Smoker

```
data.shape
```

```
(36, 4)
```

```
data['Diseased'] = data['Disease'].map({'diseased': 1, 'not diseased': 0})
data['Gender_Male'] = data['Gender'].map({'Male': 1, 'Female': 0})
data['Is_Smoker'] = data['Smoker_status'].map({'Smoker': 1, 'Non-smoker': 0})
```

```
data.head()
```

	Disease	Age	Gender	Smoker_status	Diseased	Gender_Male	Is_Smoker
0	diseased	43	Male	Smoker	1	1	1
1	not diseased	18	Male	Smoker	0	1	1
2	diseased	22	Female	Non-smoker	1	0	0
3	diseased	25	Male	Non-smoker	1	1	0
4	not diseased	45	Female	Smoker	0	0	1

```
X = data[['Age', 'Gender_Male', 'Is_Smoker']]
y = data['Diseased']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

▾ LogisticRegression ⓘ ?

▸ Parameters

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Disease Status")
print("-----------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual Diseased': y_test.values,
    'Predicted Diseased': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Disease Status
---------------------------
   Actual Diseased  Predicted Diseased
0                0                   0
1                1                   1
2                1                   0
3                1                   1
4                0                   0
5                1                   1
6                1                   1
7                1                   0
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print(" -----------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
---------------------------
Accuracy: 0.7500
Confusion Matrix:
[[2 0]
 [2 4]]
Classification Report:
              precision    recall  f1-score   support

           0       0.50      1.00      0.67         2
           1       1.00      0.67      0.80         6

    accuracy                           0.75         8
   macro avg       0.75      0.83      0.73         8
weighted avg       0.88      0.75      0.77         8
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
data = pd.read_csv('03-decision-tree-dataset-task-1.csv')
```

```
data.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
data.shape
```

```
(400, 5)
```

```
print(f"People over 30 who purchased: {data[data['Age'] > 30]['Purchased'].mean():.0%}")
```

```
People over 30 who purchased: 47%
```

```
X = data[['Age', 'EstimatedSalary']]
y = data['Purchased']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
model.fit(X_train, y_train)
```

▾ DecisionTreeClassifier ⓘ ?

▸ Parameters

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Purchased")
print("----------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Purchased
----------------------------
    Actual  Predicted
0        0          1
1        1          1
2        0          0
3        1          1
```

```
4          0           0
..       ...         ...
75         0           0
76         1           1
77         1           1
78         0           0
79         1           0

[80 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print("----------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
----------------------------
Accuracy: 0.9125
Confusion Matrix:
[[47  5]
 [ 2 26]]
Classification Report:
              precision    recall  f1-score   support

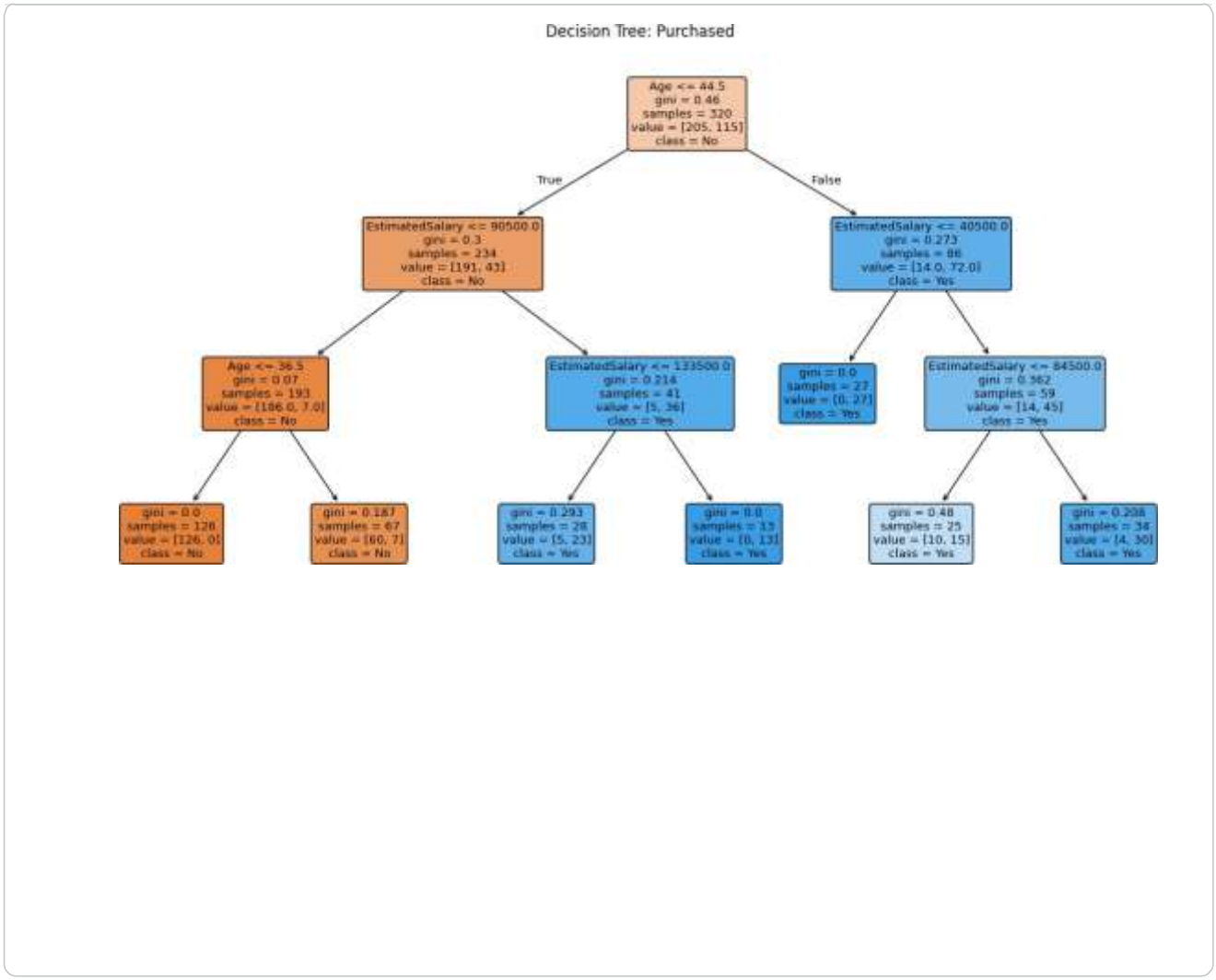
           0       0.96      0.90      0.93        52
           1       0.84      0.93      0.88        28

    accuracy                           0.91        80
   macro avg       0.90      0.92      0.91        80
weighted avg       0.92      0.91      0.91        80
```

```
print(export_text(model, feature_names=list(X.columns)))
```

```
|--- Age <= 44.50
|   |--- EstimatedSalary <= 90500.00
|   |   |--- Age <= 36.50
|   |   |   |--- class: 0
|   |   |--- Age > 36.50
|   |   |   |--- class: 0
|   |--- EstimatedSalary > 90500.00
|   |   |--- EstimatedSalary <= 133500.00
|   |   |   |--- class: 1
|   |   |--- EstimatedSalary > 133500.00
|   |   |   |--- class: 1
|--- Age > 44.50
|   |--- EstimatedSalary <= 40500.00
|   |   |--- class: 1
|   |--- EstimatedSalary > 40500.00
|   |   |--- EstimatedSalary <= 84500.00
|   |   |   |--- class: 1
|   |   |--- EstimatedSalary > 84500.00
|   |   |   |--- class: 1
```

```
plt.figure(figsize=(16, 8))
plot_tree(model, feature_names=X.columns, class_names=['No', 'Yes'], filled=True, rounded=Tru
plt.title('Decision Tree: Purchased')
plt.show()
```



The root node splits on **Age**, which makes the data purer than any other split. The next level of the tree then splits on **EstimatedSalary** (income).

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

The dataset was generated randomly: X1 is drawn from [-3, 3] and X2 from [-3, 8]. A point is labelled Class 1 when it lies above the quadratic curve X2 = X1² − 2 (with Gaussian noise), otherwise Class 0.

```
data = pd.read_csv('04-svm-dataset-task-1.csv')
```

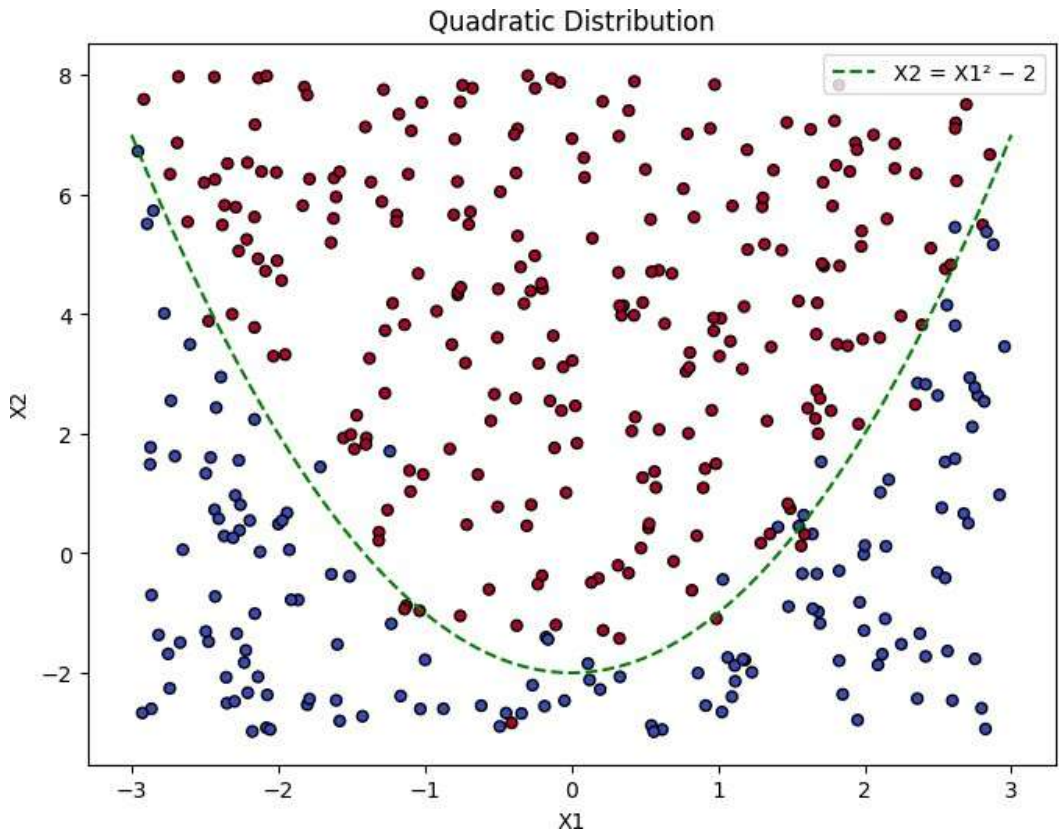
```
data.head()
```

	X1	X2	Class
0	1.644	0.321	0
1	-0.367	7.103	1
2	2.152	5.594	1
3	1.184	-1.784	0
4	-2.435	7.967	1

```
data.shape
```

```
(400, 3)
```

```
xs = np.linspace(-3, 3, 200)
plt.figure(figsize=(8, 6))
plt.scatter(data['X1'], data['X2'], c=data['Class'], cmap='coolwarm', edgecolors='k', s=25)
plt.plot(xs, xs**2 - 2, 'g--', label='X2 = X1² − 2')
plt.xlabel('X1'); plt.ylabel('X2'); plt.title('Quadratic Distribution'); plt.legend()
plt.show()
```




```
X = data[['X1', 'X2']]
y = data['Class']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

A linear kernel can only draw a straight line, so it can't follow the curve. It's shown here for comparison.

```
linear_model = make_pipeline(StandardScaler(), SVC(kernel='linear'))
linear_model.fit(X_train, y_train)
print(f"Linear kernel accuracy: {accuracy_score(y_test, linear_model.predict(X_test)):.4f}")
```

```
Linear kernel accuracy: 0.7875
```

A polynomial kernel with degree 2 can learn a quadratic decision boundary. coef0=1 adds the linear and constant terms, so the kernel covers the full X2 = aX1² + bX1 + c form.

```
model = make_pipeline(StandardScaler(), SVC(kernel='poly', degree=2, coef0=1, C=10))
model.fit(X_train, y_train)
```

```
▸ Pipeline
▸ StandardScaler
▸ SVC
```

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Class")
print("-------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': predictions
```

```
})
print(comparison_df)
```

```
Actual vs Predicted Class
-------------------------------------------------
    Actual  Predicted
0        1          1
1        0          0
2        1          1
3        1          1
4        1          1
..     ...        ...
75       0          0
76       0          0
77       1          1
78       1          1
79       0          0

[80 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print("-------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

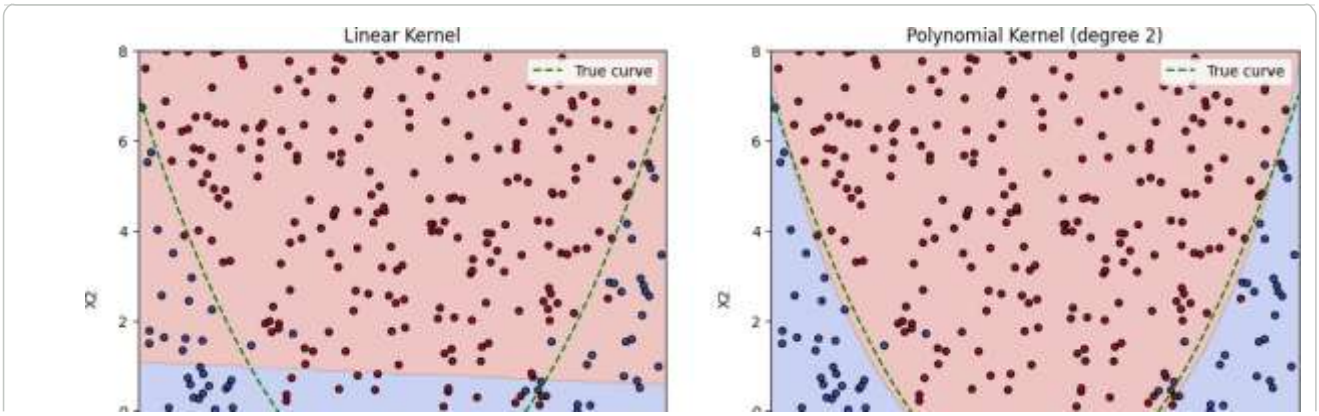
```
Classification Evaluation Metrics
----------------------------
Accuracy: 0.9250
Confusion Matrix:
[[29  4]
 [ 2 45]]
Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.88      0.91        33
           1       0.92      0.96      0.94        47

    accuracy                           0.93        80
   macro avg       0.93      0.92      0.92        80
weighted avg       0.93      0.93      0.92        80
```

```
xx, yy = np.meshgrid(np.linspace(-3, 3, 300), np.linspace(-3, 8, 300))
grid = pd.DataFrame({'X1': xx.ravel(), 'X2': yy.ravel()})

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, m, title in [(axes[0], linear_model, 'Linear Kernel'), (axes[1], model, 'Polynomial K
    zz = m.predict(grid).reshape(xx.shape)
    ax.contourf(xx, yy, zz, alpha=0.3, cmap='coolwarm')
    ax.scatter(data['X1'], data['X2'], c=data['Class'], cmap='coolwarm', edgecolors='k', s=20
    ax.plot(xs, xs**2 - 2, 'g--', label='True curve')
    ax.set_title(title); ax.set_xlabel('X1'); ax.set_ylabel('X2'); ax.legend()
plt.show()
```


Linear Kernel
True curve
X2
Polynomial Kernel (degree 2)
True curve
X2

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Synthetic email dataset. Freq_* columns are the percentage of words in the email that match that word, Exclamation_Count is the number of '!' characters and Message_Length is the number of characters. Spam = 1 means spam, 0 means not spam.

```
data = pd.read_csv('04-svm-dataset-task-2.csv')
```

```
data.head()
```

	Freq_Free	Freq_Money	Freq_Win	Freq_Meeting	Exclamation_Count	Message_Length	Spam
0	0.62	0.30	0.23	0.17	0	282	0
1	1.09	0.00	0.00	0.00	2	634	0
2	0.80	0.46	0.00	0.44	5	315	1
3	0.03	0.00	0.13	0.86	3	318	0
4	0.07	0.00	0.28	1.34	1	403	0

```
data.shape
```

```
(500, 7)
```

```
data['Spam'].value_counts()
```

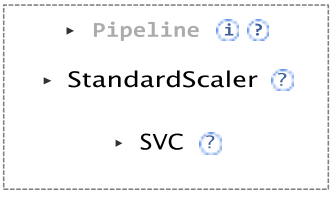
```
Spam
0    302
1    198
Name: count, dtype: int64
```

```
X = data.drop(columns='Spam')
y = data['Spam']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, str
```

SVMs are sensitive to feature scale, and Message_Length is in the hundreds while word frequencies are below 3. StandardScaler brings them to the same scale before the RBF-kernel SVM.

```
model = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1, gamma='scale'))
model.fit(X_train, y_train)
```



```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Spam")
print("--------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Spam
--------------------------------------------------
    Actual  Predicted
0        0          0
1        1          1
2        0          0
3        0          0
4        1          1
..     ...        ...
95       1          1
96       1          0
97       1          0
98       0          0
99       1          1

[100 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print("--------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
----------------------------
Accuracy: 0.8700
Confusion Matrix:
[[57  3]
 [10 30]]
Classification Report:
              precision    recall  f1-score   support

           0       0.85      0.95      0.90        60
           1       0.91      0.75      0.82        40

    accuracy                           0.87       100
   macro avg       0.88      0.85      0.86       100
weighted avg       0.87      0.87      0.87       100
```

```
new_emails = pd.DataFrame([
    {'Freq_Free': 1.5, 'Freq_Money': 1.0, 'Freq_Win': 0.8, 'Freq_Meeting': 0.0, 'Exclamation_
    {'Freq_Free': 0.0, 'Freq_Money': 0.1, 'Freq_Win': 0.0, 'Freq_Meeting': 0.9, 'Exclamation_
])
new_emails.assign(Prediction=np.where(model.predict(new_emails) == 1, 'Spam', 'Not Spam'))
```

	Freq_Free	Freq_Money	Freq_Win	Freq_Meeting	Exclamation_Count	Message_Length	Prediction
0	1.5	1.0	0.8	0.0	6	150	Spam
1	0.0	0.1	0.0	0.9	0	480	Not Spam

Name GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import zipfile
import numpy as np
import pandas as pd
from collections import Counter
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Dataset: `Ch02/digits.zip` from pbharrin/machinelearninginaction. Each handwriflen digit is a 32x32 grid of 0s and 1s stored as a text file named `<digit>_<index>.txt`, split into `trainingDigits/` and `testDigits/`.

```
def img2vector(text):
    # flatten a 32x32 text image into a 1x1024 vector
    return np.array([int(ch) for line in text.split() for ch in line[:32]])

def load_digits(zip_path, folder):
    X, y = [], []
    with zipfile.ZipFile(zip_path) as z:
        for name in z.namelist():
            if name.startswith(folder + '/') and name.endswith('.txt'):
                X.append(img2vector(z.read(name).decode()))
                y.append(int(name.split('/')[1].split('_')[0]))
    return np.array(X), np.array(y)
```

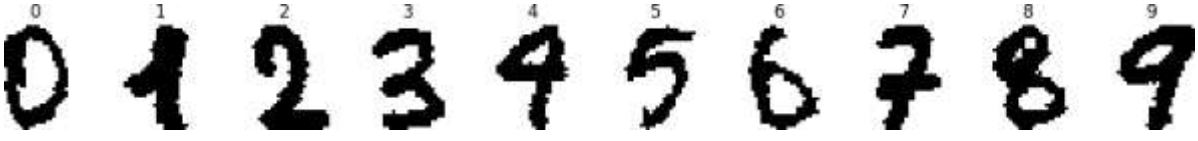
```
X_train, y_train = load_digits('digits.zip', 'trainingDigits')
X_test, y_test = load_digits('digits.zip', 'testDigits')
print("Training samples:", X_train.shape)
print("Test samples:", X_test.shape)
```

```
Training samples: (1934, 1024)
Test samples: (946, 1024)
```

```
pd.Series(y_train).value_counts().sort_index()
```

```
0    189
1    198
2    195
3    199
4    186
5    187
6    195
7    201
8    180
9    204
Name: count, dtype: int64
```

```
fig, axes = plt.subplots(1, 10, figsize=(15, 2))
for digit, ax in enumerate(axes):
    ax.imshow(X_train[y_train == digit][0].reshape(32, 32), cmap='gray_r')
    ax.set_title(digit); ax.axis('off')
plt.show()
```



kNN classifier from the book (`classify0`), ported to Python 3: compute the Euclidean distance to every training sample, take the k nearest, and return the majority label.

```
def classify0(inX, dataSet, labels, k):
    distances = np.sqrt(((dataSet - inX) ** 2).sum(axis=1))
    nearest = labels[distances.argsort()[:k]]
    return Counter(nearest).most_common(1)[0][0]
```

```
predictions = np.array([classify0(x, X_train, y_train, 3) for x in X_test])
```

```
print("Actual vs Predicted Digit")
print("--------------------------------------------------")
comparison_df = pd.DataFrame({
    'Actual': y_test,
    'Predicted': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Digit
--------------------------------------------------
     Actual  Predicted
0         0          0
1         0          0
2         0          0
3         0          0
4         0          0
..      ...        ...
941       9          9
942       9          9
943       9          9
944       9          9
945       9          9

[946 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

print("Classification Evaluation Metrics")
print("--------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print(f"Errors: {(predictions != y_test).sum()} out of {len(y_test)}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
--------------------------------------------------
Accuracy: 0.9884
Errors: 11 out of 946
Confusion Matrix:
[[ 87   0   0   0   0   0   0   0   0   0]
 [  0  96   0   0   0   0   0   1   0   0]
 [  0   0  92   0   0   0   0   0   0   0]
 [  0   0   0  84   0   0   0   0   0   1]
 [  0   0   0   0 114   0   0   0   0   0]
 [  0   0   0   1   0 106   1   0   0   0]
 [  0   0   0   0   0   0  87   0   0   0]
 [  0   0   0   0   0   0   0  96   0   0]
 [  0   2   0   1   0   0   1   0  87   0]
 [  0   1   0   0   0   1   0   1   0  86]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        87
           1       0.97      0.99      0.98        97
           2       1.00      1.00      1.00        92
           3       0.98      0.99      0.98        85
           4       1.00      1.00      1.00       114
           5       0.99      0.98      0.99       108
```

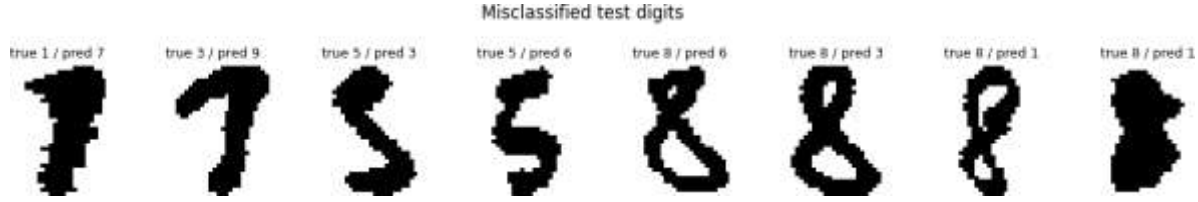
6	0.98	1.00	0.99	87
7	0.98	1.00	0.99	96
8	1.00	0.96	0.98	91
9	0.99	0.97	0.98	89
accuracy			0.99	946
macro avg	0.99	0.99	0.99	946
weighted avg	0.99	0.99	0.99	946

The same model using scikit-learn's `KNeighborsClassifier` (k=3, Euclidean distance) for comparison.

```
model = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
model.fit(X_train, y_train)
print(f"scikit-learn kNN accuracy: {accuracy_score(y_test, model.predict(X_test)):.4f}")
```

```
scikit-learn kNN accuracy: 0.9884
```

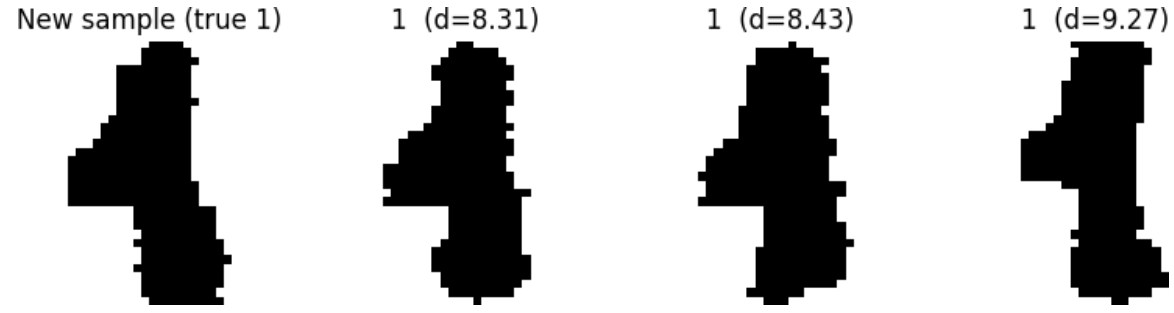
```
wrong = np.where(predictions != y_test)[0]
fig, axes = plt.subplots(1, len(wrong[:8]), figsize=(15, 2.5))
for ax, i in zip(np.atleast_1d(axes), wrong[:8]):
    ax.imshow(X_test[i].reshape(32, 32), cmap='gray_r')
    ax.set_title(f"true {y_test[i]} / pred {predictions[i]}", fontsize=9); ax.axis('off')
plt.suptitle('Misclassified test digits')
plt.show()
```



Identifying a new handwriflen sample: pick one test image the classifier has never seen and find its nearest neighbours.

```
new_digit, true_label = X_test[100], y_test[100]
distances = np.sqrt(((X_train - new_digit) ** 2).sum(axis=1))
nearest = distances.argsort()[:3]

fig, axes = plt.subplots(1, 4, figsize=(10, 2.5))
axes[0].imshow(new_digit.reshape(32, 32), cmap='gray_r'); axes[0].set_title(f"New sample (tru
for ax, i in zip(axes[1:], nearest):
    ax.imshow(X_train[i].reshape(32, 32), cmap='gray_r'); ax.set_title(f"{y_train[i]} (d={di
for ax in axes: ax.axis('off')
plt.show()
print("Predicted digit:", classify0(new_digit, X_train, y_train, 3))
```




```
Predicted digit: 1
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
```

```
data = pd.read_csv('05-knn-dataset-task-2.csv')
data
```

	Brightness	Saturation	Class
0	40	20	Red
1	50	50	Blue
2	60	90	Blue
3	10	25	Red
4	70	70	Blue
5	60	10	Red
6	25	80	Blue

New entry to classify: **Brightness = 20**, **Saturation = 35**.

Euclidean distance between the new entry (x_1, y_1) and an existing point (x_2, y_2):

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

```
new_entry = {'Brightness': 20, 'Saturation': 35}
```

```
data['Distance'] = np.sqrt((data['Brightness'] - new_entry['Brightness']) ** 2 +
                           (data['Saturation'] - new_entry['Saturation']) ** 2).round(2)
for _, row in data.iterrows():
    print(f"d = sqrt(({row['Brightness']} - 20)^2 + ({row['Saturation']} - 35)^2) = {row['Dis
```

```
d = sqrt((40 - 20)^2 + (20 - 35)^2) = 25.0
d = sqrt((50 - 20)^2 + (50 - 35)^2) = 33.54
d = sqrt((60 - 20)^2 + (90 - 35)^2) = 68.01
d = sqrt((10 - 20)^2 + (25 - 35)^2) = 14.14
d = sqrt((70 - 20)^2 + (70 - 35)^2) = 61.03
d = sqrt((60 - 20)^2 + (10 - 35)^2) = 47.17
d = sqrt((25 - 20)^2 + (80 - 35)^2) = 45.28
```

```
ranked = data.sort_values('Distance').reset_index(drop=True)
ranked.index += 1
ranked
```

	Brightness	Saturation	Class	Distance
1	10	25	Red	14.14
2	40	20	Red	25.00
3	50	50	Blue	33.54
4	25	80	Blue	45.28
5	60	10	Red	47.17
6	70	70	Blue	61.03
7	60	90	Blue	68.01

```
k = 5
nearest = ranked.head(k)
votes = nearest['Class'].value_counts()
print(f"Votes from the {k} nearest neighbours:")
print(votes)
print("New entry is classified as:", votes.idxmax())
```

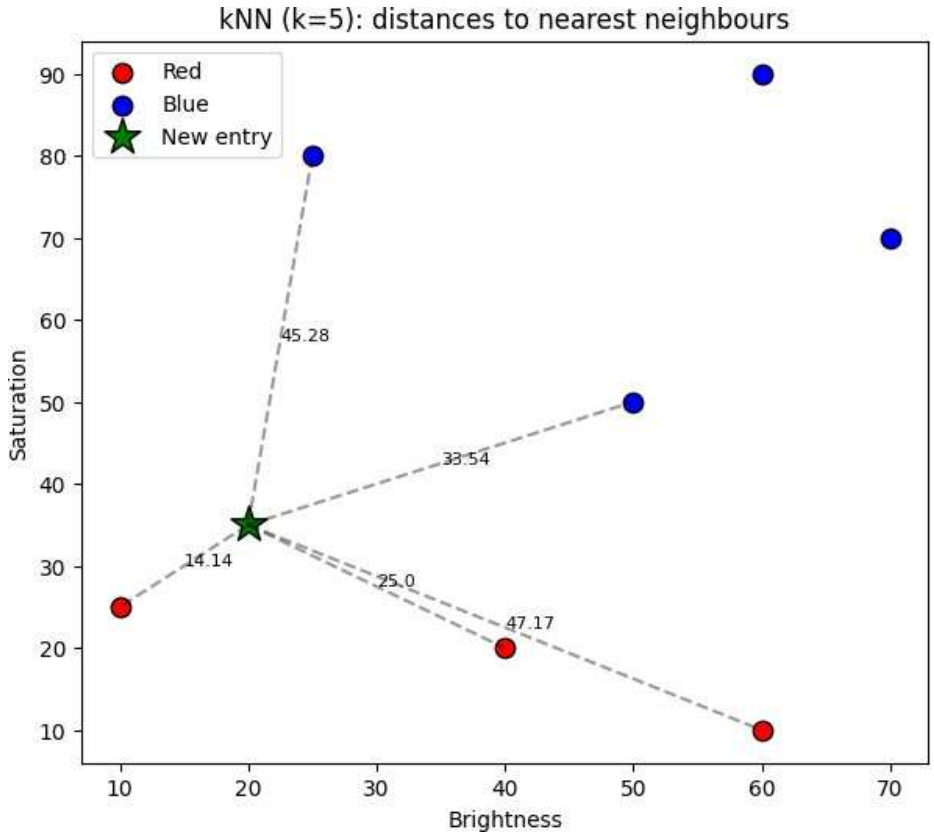
```
Votes from the 5 nearest neighbours:
Class
Red     3
Blue    2
Name: count, dtype: int64
New entry is classified as: Red
```

Checking the result with scikit-learn's `KNeighborsClassifier` (k=5, Euclidean distance):

```
model = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
model.fit(data[['Brightness', 'Saturation']], data['Class'])
print("scikit-learn prediction:", model.predict(pd.DataFrame([new_entry]))[0])
```

```
scikit-learn prediction: Red
```

```
plt.figure(figsize=(7, 6))
for cls, colour in [('Red', 'red'), ('Blue', 'blue')]:
    pts = data[data['Class'] == cls]
    plt.scatter(pts['Brightness'], pts['Saturation'], c=colour, s=80, label=cls, edgecolors='
for _, row in nearest.iterrows():
    plt.plot([20, row['Brightness']], [35, row['Saturation']], 'k--', alpha=0.4)
    plt.annotate(row['Distance'], ((20 + row['Brightness']) / 2, (35 + row['Saturation']) / 2
plt.scatter(20, 35, c='green', marker='*', s=300, label='New entry', edgecolors='k')
plt.xlabel('Brightness'); plt.ylabel('Saturation'); plt.title(f'kNN (k={k}): distances to nea
plt.legend()
plt.show()
```



Name GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
```

Dataset: Old Faithful geyser, Yellowstone National Park, Wyoming, USA (R `datasets::faithful`, 272 observations). `eruptions` is the eruption duration in minutes and `waiting` is the waiting time to the next eruption in minutes.

```
data = pd.read_csv('06-kmeans-dataset-task-1.csv')
```

```
data.head()
```

	eruptions	waiting
0	3.600	79
1	1.800	54
2	3.333	74
3	2.283	62
4	4.533	85

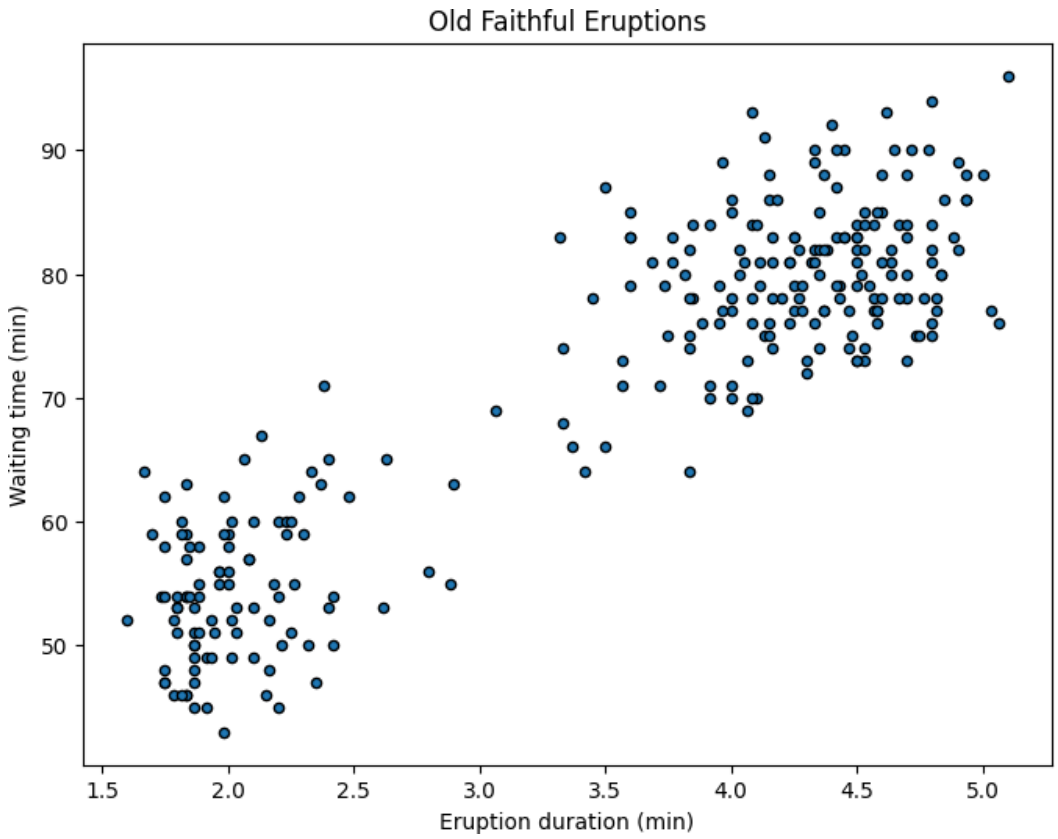
```
data.shape
```

```
(272, 2)
```

```
data.describe()
```

	eruptions	waiting
count	272.000000	272.000000
mean	3.487783	70.897059
std	1.141371	13.594974
min	1.600000	43.000000
25%	2.162750	58.000000
50%	4.000000	76.000000
75%	4.454250	82.000000
max	5.100000	96.000000

```
plt.figure(figsize=(8, 6))
plt.scatter(data['eruptions'], data['waiting'], s=20, edgecolors='k')
plt.xlabel('Eruption duration (min)'); plt.ylabel('Waiting time (min)')
plt.title('Old Faithful Eruptions')
plt.show()
```



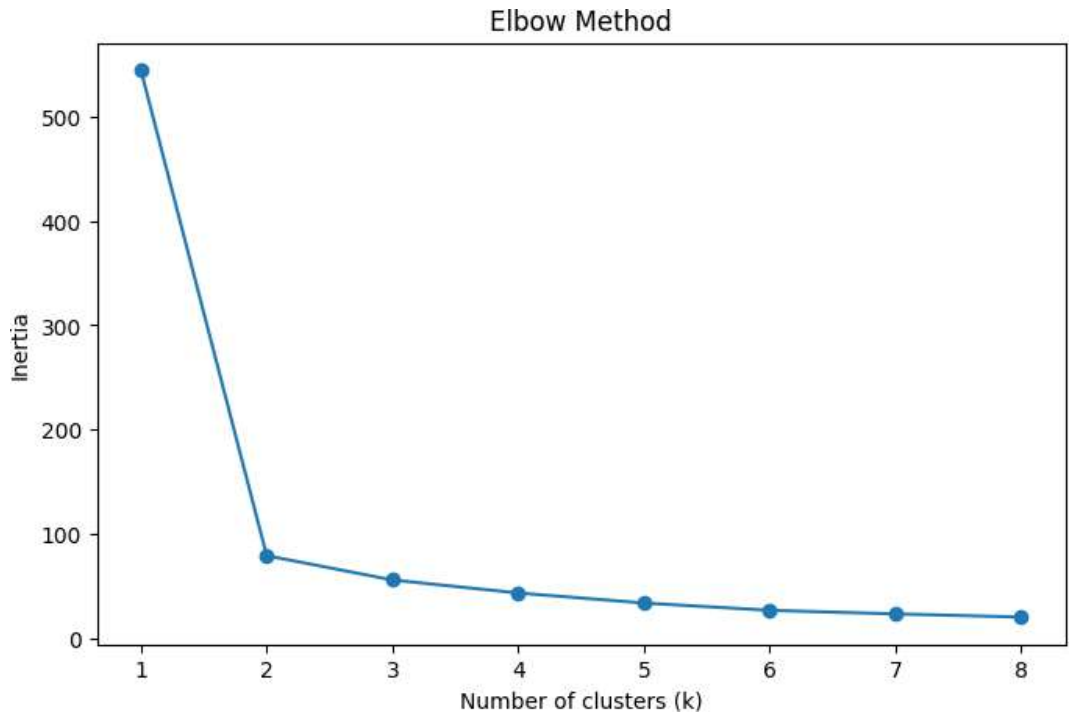
The two features are on different scales (eruptions is 1.6–5.1 and waiting is 43–96), so they are standardised first. Without this, waiting time would dominate the distance calculation.

```
X = data[['eruptions', 'waiting']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Elbow method: inertia (within-cluster sum of squares) for k = 1 to 8.

```
inertias = [KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled).inertia_ for k in

plt.figure(figsize=(8, 5))
plt.plot(range(1, 9), inertias, 'o-')
plt.xlabel('Number of clusters (k)'); plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```




```
for k in range(2, 7):
    labels = KMeans(n_clusters=k, n_init=10, random_state=42).fit_predict(X_scaled)
    print(f"k = {k}: silhouette score = {silhouette_score(X_scaled, labels):.4f}")
```

```
k = 2: silhouette score = 0.7452
k = 3: silhouette score = 0.4851
k = 4: silhouette score = 0.3822
k = 5: silhouette score = 0.3628
k = 6: silhouette score = 0.3920
```

The elbow is at k = 2, and k = 2 also has the highest silhouefle score, so the eruptions are segmented into 2 clusters.

```
model = KMeans(n_clusters=2, n_init=10, random_state=42)
data['Cluster'] = model.fit_predict(X_scaled)
```

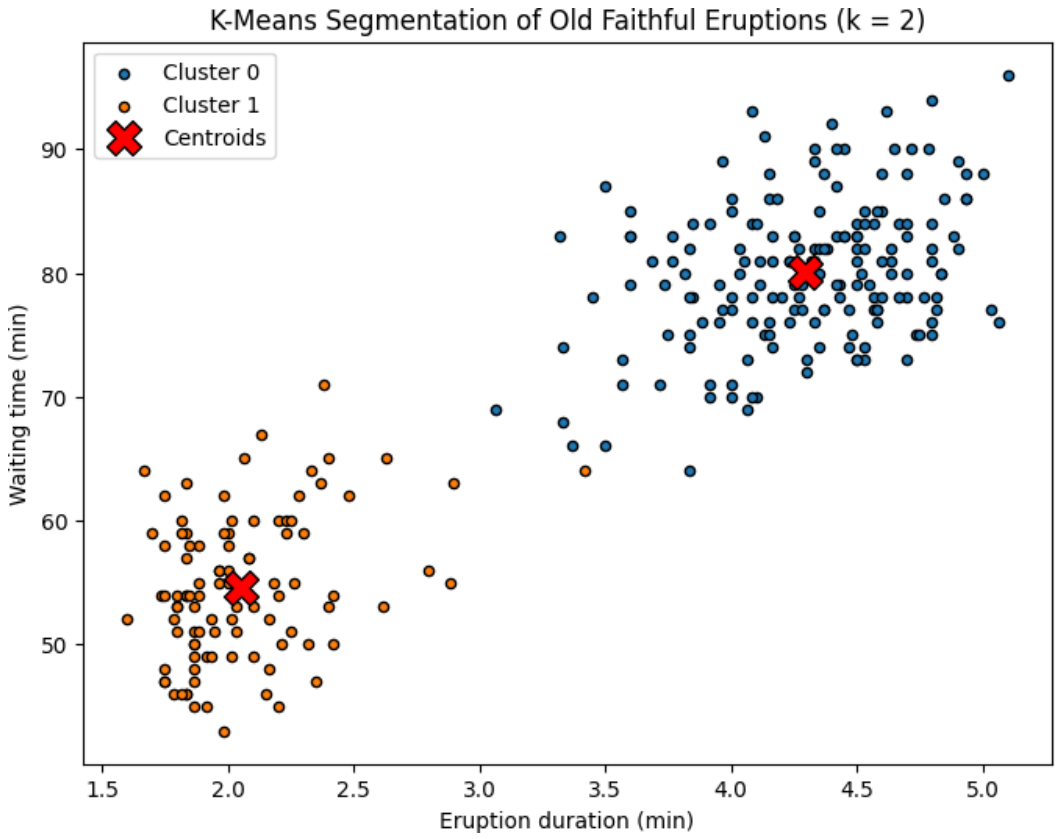
```
centres = pd.DataFrame(scaler.inverse_transform(model.cluster_centers_), columns=['eruptions'
centres.index.name = 'Cluster'
print("Cluster centres (original units, minutes)")
print("----------")
print(centres.round(2))
```

```
Cluster centres (original units, minutes)
----------
         eruptions  waiting
Cluster
0             4.30    80.08
1             2.05    54.59
```

```
print("Cluster summary")
print("----------")
print(data.groupby('Cluster')[['eruptions', 'waiting']].agg(['count', 'mean', 'min', 'max']).
```

```
Cluster summary
----------
         eruptions                    waiting
             count  mean   min   max   count   mean min max
Cluster
0              174  4.30  3.07  5.10     174  80.08  64  96
1               98  2.05  1.60  3.42      98  54.59  43  71
```

```
plt.figure(figsize=(8, 6))
for cluster, colour in [(0, 'tab:blue'), (1, 'tab:orange')]:
    pts = data[data['Cluster'] == cluster]
    plt.scatter(pts['eruptions'], pts['waiting'], c=colour, s=20, edgecolors='k', label=f'Clu
plt.scatter(centres['eruptions'], centres['waiting'], c='red', marker='X', s=250, edgecolors=
plt.xlabel('Eruption duration (min)'); plt.ylabel('Waiting time (min)')
plt.title('K-Means Segmentation of Old Faithful Eruptions (k = 2)')
plt.legend()
plt.show()
```




```
short = centres['eruptions'].idxmin()
```

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

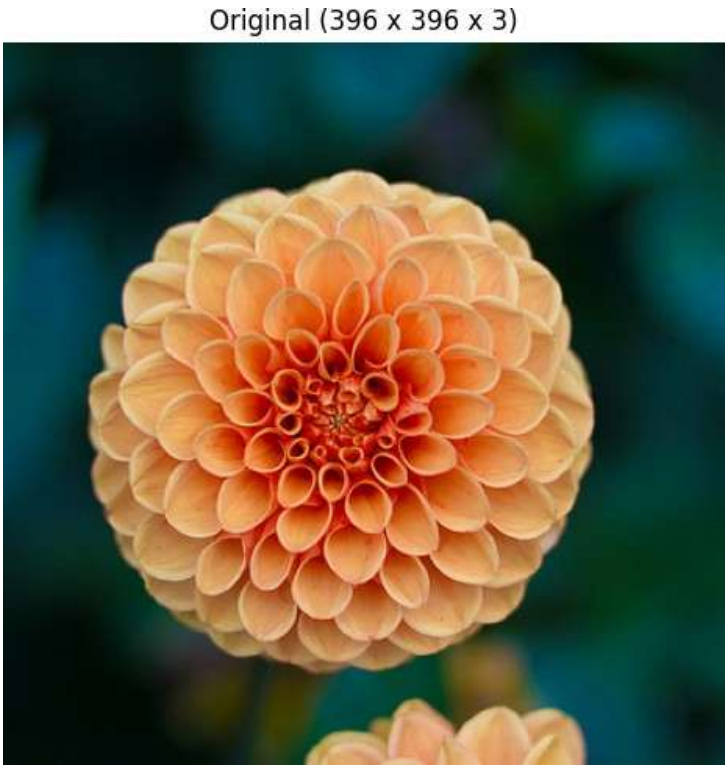
```
import numpy as np
from PIL import Image
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

The image is a 396 x 396 x 3 centre crop of scikit-learn's sample `flower.jpg`, saved as a lossless PNG. Each pixel is stored as 3 colour values (R, G, B) of 8 bits each, so the image can use up to $2^{24} \approx 16.7$ million colours. K-Means compresses it by grouping all pixels into K colours and storing only a K-colour palefle plus a small palefle index for each pixel.

```
image = np.array(Image.open('06-kmeans-image-task-2.png'))
print("Image shape:", image.shape)
print("Unique colours:", len(np.unique(image.reshape(-1, 3), axis=0)))
```

```
Image shape: (396, 396, 3)
Unique colours: 56775
```

```
plt.figure(figsize=(6, 6))
plt.imshow(image); plt.axis('off'); plt.title('Original (396 x 396 x 3)')
plt.show()
```




```
pixels = image.reshape(-1, 3) / 255.0
print("Pixels as data points:", pixels.shape)
```

```
Pixels as data points: (156816, 3)
```

```
K = 16
model = KMeans(n_clusters=K, n_init=4, random_state=42)
labels = model.fit_predict(pixels)
palette = model.cluster_centers_
```

```
compressed = (palette[labels].reshape(image.shape) * 255).astype(np.uint8)
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(image); axes[0].set_title('Original')
axes[1].imshow(compressed); axes[1].set_title(f'Compressed (K = {K} colours)')
for ax in axes: ax.axis('off')
plt.show()
```

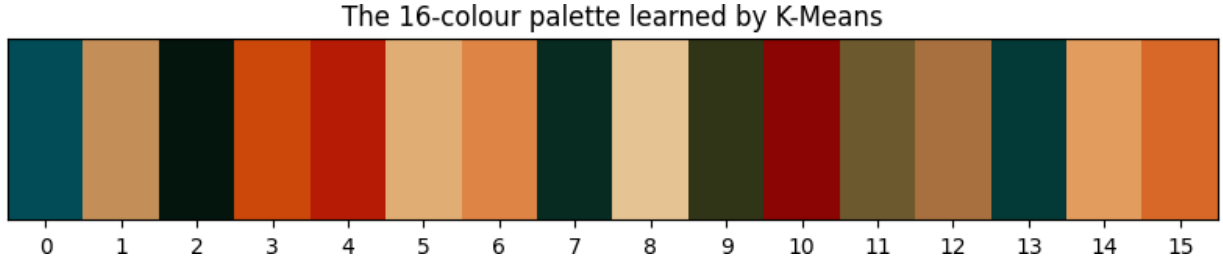



```
h, w, _ = image.shape
original_bits = h * w * 24
compressed_bits = h * w * int(np.ceil(np.log2(K))) + K * 24

print("Compression Summary")
print("----------------------------------------------------")
print(f"Original size:      {original_bits:,} bits ({original_bits / 8 / 1024:.1f} KB)")
print(f"Compressed size: {compressed_bits:,} bits ({compressed_bits / 8 / 1024:.1f} KB)")
print(f"Compression ratio: {original_bits / compressed_bits:.2f}x")
```

```
Compression Summary
---------------------------
Original size:     3,763,584 bits (459.4 KB)
Compressed size: 627,648 bits (76.6 KB)
Compression ratio: 6.00x
```

```
plt.figure(figsize=(10, 1.5))
plt.imshow(palette.reshape(1, K, 3), aspect='auto')
plt.yticks([]); plt.xticks(range(K)); plt.title(f'The {K}-colour palette learned by K-Means')
plt.show()
```



Comparing different values of K: fewer colours give a smaller image but lose more detail.

```
ks = [2, 4, 8, 16, 32]
fig, axes = plt.subplots(1, len(ks), figsize=(20, 4.5))
for ax, k in zip(axes, ks):
    km = KMeans(n_clusters=k, n_init=4, random_state=42).fit(pixels)
    img_k = km.cluster_centers_[km.labels_].reshape(image.shape)
    bits = h * w * int(np.ceil(np.log2(k))) + k * 24
    ax.imshow(img_k); ax.axis('off')
    ax.set_title(f"K = {k}\n{original_bits / bits:.1f}x smaller")
plt.show()
```



Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
```

Problem: Maggie, a real-estate agent, wants to know why some of the properties her company manages have not sold for more than six months.

Dataset: a synthetic set of 500 property listings with pricing, size, condition, location and marketing features. `Days_on_Market` is how long each property has been listed, and `Unsold_6_Months` = 1 means it has been on the market for more than 180 days.

```
data = pd.read_csv('07-pca-dataset-task-1.csv')
```

```
data.head()
```

	Property_ID	List_Price	Market_Value	Price_Premium_%	Area_sqft	Bedrooms	Bathrooms	Age_Y
0	P001	346000	307000	12.8	1469	3	2	
1	P002	145000	164000	-11.4	857	2	1	
2	P003	277000	303000	-8.5	1897	4	3	
3	P004	283000	288000	-1.6	1863	4	3	
4	P005	60000	58000	3.8	500	1	1	

```
data.shape
```

```
(500, 15)
```

```
data['Unsold_6_Months'].value_counts()
```

```
Unsold_6_Months
0    366
1    134
Name: count, dtype: int64
```

```
features = ['List_Price', 'Market_Value', 'Price_Premium_%', 'Area_sqft', 'Bedrooms', 'Bathro
            'Age_Years', 'Condition_Score', 'Crime_Rate', 'School_Rating', 'Distance_to_City_
X = data[features]
X_scaled = StandardScaler().fit_transform(X)
```

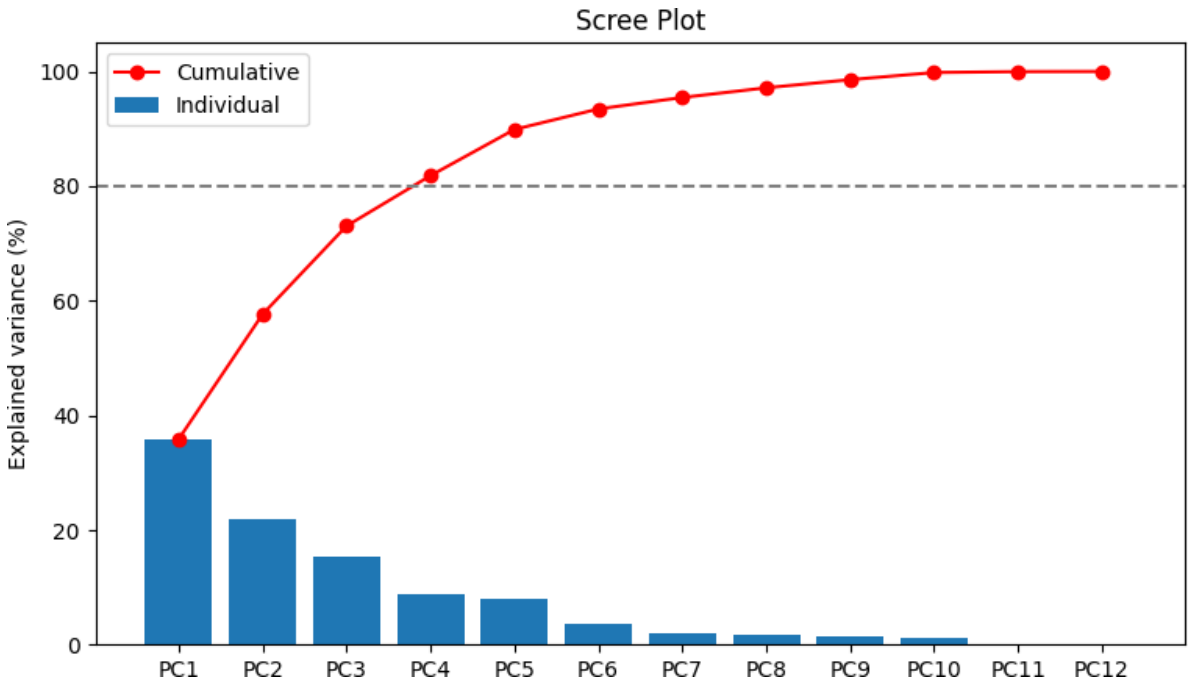
PCA on the 12 standardised property features (the outcome columns are left out, so PCA only finds structure in the properties themselves).

```
pca = PCA()
scores = pca.fit_transform(X_scaled)

explained = pd.DataFrame({
    'Explained Variance %': (pca.explained_variance_ratio_ * 100).round(2),
    'Cumulative %': (pca.explained_variance_ratio_.cumsum() * 100).round(2),
}, index=[f'PC{i}' for i in range(1, len(features) + 1)])
explained
```

	Explained Variance %	Cumulative %
PC1	35.83	35.83
PC2	21.80	57.63
PC3	15.40	73.04
PC4	8.79	81.82
PC5	8.06	89.88
PC6	3.57	93.46
PC7	2.00	95.46
PC8	1.69	97.15
PC9	1.41	98.57
PC10	1.25	99.82
PC11	0.16	99.97
PC12	0.03	100.00

```
plt.figure(figsize=(9, 5))
plt.bar(explained.index, explained['Explained Variance %'], label='Individual')
plt.plot(explained.index, explained['Cumulative %'], 'ro-', label='Cumulative')
plt.axhline(80, color='grey', linestyle='--')
plt.ylabel('Explained variance (%)'); plt.title('Scree Plot')
plt.legend()
plt.show()
```




```
n_pc = int((explained['Cumulative %'] < 80).sum()) + 1
print(f"{n_pc} components explain at least 80% of the variance")
```

```
4 components explain at least 80% of the variance
```

Loadings: how strongly each original feature contributes to each principal component. Large absolute values show what a component represents.

```
loadings = pd.DataFrame(pca.components_[:n_pc].T, index=features,
                        columns=[f'PC{i}' for i in range(1, n_pc + 1)]).round(3)
loadings.style.background_gradient(cmap='RdBu_r', vmin=-0.7, vmax=0.7)
```

	PC1	PC2	PC3	PC4
List_Price	0.466000	0.024000	-0.009000	0.171000
Market_Value	0.471000	0.024000	-0.040000	-0.025000
Price_Premium_%	0.060000	0.010000	0.116000	0.826000
Area_sqft	0.429000	0.226000	0.088000	-0.025000
Bedrooms	0.389000	0.204000	0.098000	-0.082000
Bathrooms	0.353000	0.178000	0.103000	-0.110000
Age_Years	-0.072000	-0.049000	0.689000	-0.011000
Condition_Score	0.073000	0.044000	-0.689000	0.033000
Crime_Rate	-0.168000	0.544000	0.012000	0.036000
School_Rating	0.168000	-0.534000	0.005000	-0.011000
Distance_to_City_km	-0.165000	0.537000	-0.001000	0.028000
Listing_Photos	-0.052000	-0.024000	-0.085000	0.515000

```
for pc in loadings.columns:
    top = loadings[pc].abs().sort_values(ascending=False).head(3).index
    print(f"{pc}: " + ", ".join(f"{f} ({loadings.loc[f, pc]:+.2f})" for f in top))
```

```
PC1: Market_Value (+0.47), List_Price (+0.47), Area_sqft (+0.43)
PC2: Crime_Rate (+0.54), Distance_to_City_km (+0.54), School_Rating (−0.53)
PC3: Age_Years (+0.69), Condition_Score (−0.69), Price_Premium_% (+0.12)
PC4: Price_Premium_% (+0.83), Listing_Photos (+0.52), List_Price (+0.17)
```

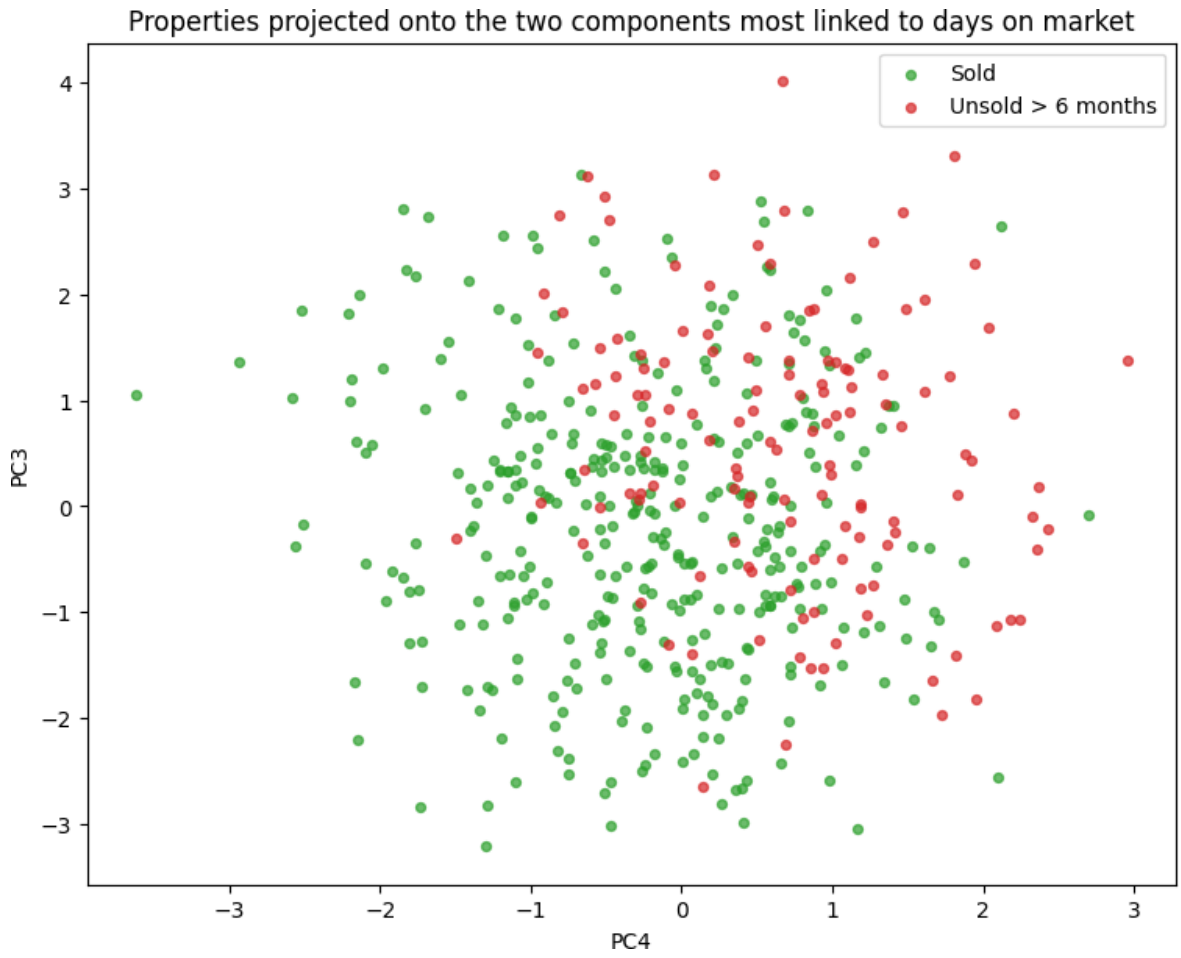
Which components separate the unsold properties? Compare the average component score of sold and unsold properties, and correlate each component with days on market.

```
pc_scores = pd.DataFrame(scores[:, :n_pc], columns=loadings.columns)
pc_scores['Unsold_6_Months'] = data['Unsold_6_Months']

comparison = pc_scores.groupby('Unsold_6_Months').mean().T
comparison.columns = ['Sold (mean score)', 'Unsold > 6 months (mean score)']
comparison['Difference'] = comparison.iloc[:, 1] - comparison.iloc[:, 0]
comparison['Correlation with Days_on_Market'] = [np.corrcoef(pc_scores[pc], data['Days_on_Mar
comparison.round(3)
```

	Sold (mean score)	Unsold > 6 months (mean score)	Difference	Correlation with Days_on_Market
PC1	0.114	-0.312	-0.427	-0.062
PC2	-0.210	0.575	0.785	0.293
PC3	-0.211	0.577	0.789	0.407
PC4	-0.246	0.671	0.917	0.544

```
key = comparison['Correlation with Days_on_Market'].abs().sort_values(ascending=False).index[
plt.figure(figsize=(9, 7))
for label, colour, name in [(0, 'tab:green', 'Sold'), (1, 'tab:red', 'Unsold > 6 months')]:
    pts = pc_scores[pc_scores['Unsold_6_Months'] == label]
    plt.scatter(pts[key[0]], pts[key[1]], c=colour, s=18, alpha=0.7, label=name)
plt.xlabel(key[0]); plt.ylabel(key[1])
plt.title('Properties projected onto the two components most linked to days on market')
plt.legend()
plt.show()
```



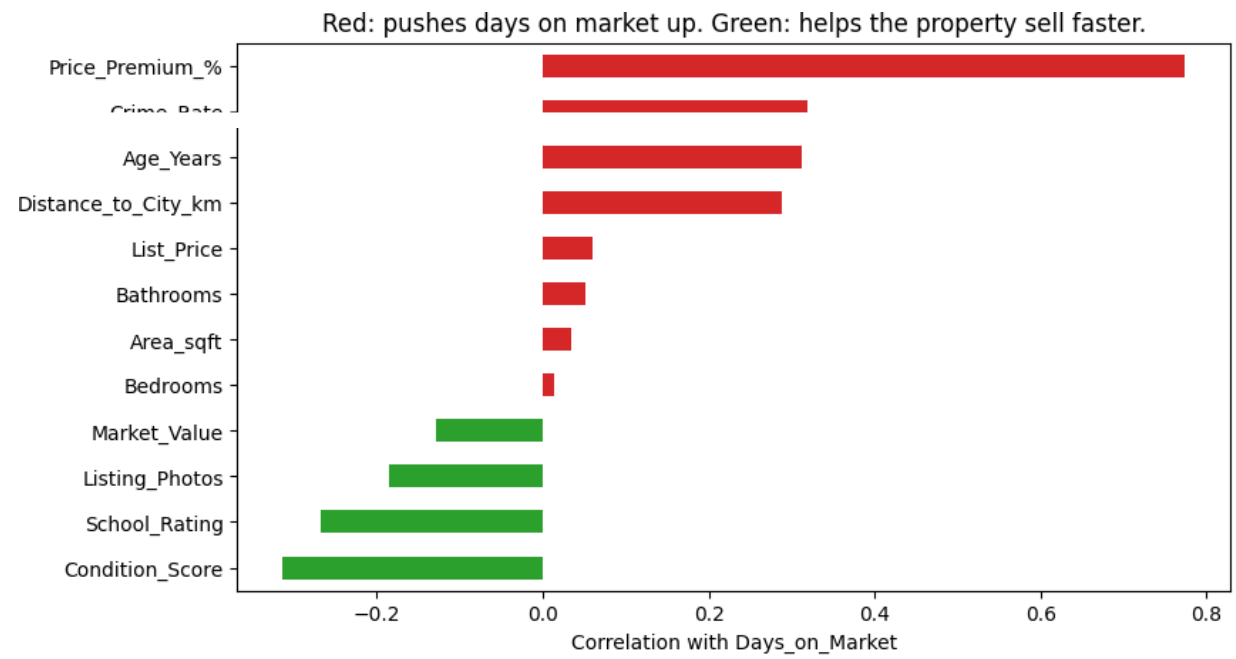
Checking the insight against the raw features: average values for sold vs unsold properties.

```
profile = data.groupby('Unsold_6_Months')[features].mean().T
profile.columns = ['Sold', 'Unsold > 6 months']
profile['Difference %'] = ((profile['Unsold > 6 months'] - profile['Sold']) / profile['Sold']
profile.round(2)
```

	Sold	Unsold > 6 months	Difference %
List_Price	260674.86	261253.73	0.22
Market_Value	253724.04	227708.96	-10.25
Price_Premium_%	2.31	14.37	522.15
Area_sqft	1383.90	1363.43	-1.48
Bedrooms	3.02	2.91	-3.69
Bathrooms	2.04	2.05	0.42
Age_Years	23.50	28.46	21.10
Condition_Score	6.25	5.38	-13.91
Crime_Rate	28.98	34.58	19.33
School_Rating	6.13	5.29	-13.72
Distance_to_City_km	11.58	13.94	20.37
Listing_Photos	23.69	20.19	-14.77

```
corr = data[features + ['Days_on_Market']].corr()['Days_on_Market'].drop('Days_on_Market').so
plt.figure(figsize=(9, 5))
corr.plot.barh(color=['tab:green' if v < 0 else 'tab:red' for v in corr])
plt.xlabel('Correlation with Days_on_Market')
```

```
plt.title('Red: pushes days on market up. Green: helps the property sell faster.')
plt.show()
```




```
print("Insights for Maggie")
print("----------------------------------------------")
for pc in comparison.index:
    c = comparison.loc[pc, 'Correlation with Days_on_Market']
    top = loadings[pc].abs().sort_values(ascending=False).head(3).index
    print(f"{pc} (corr with days on market {c:+.2f}): driven by {', '.join(top)}")
```

```
Insights for Maggie
----------------------------
PC1 (corr with days on market -0.06): driven by Market_Value, List_Price, Area_sqft
PC2 (corr with days on market +0.29): driven by Crime_Rate, Distance_to_City_km, School_Rating
PC3 (corr with days on market +0.41): driven by Age_Years, Condition_Score, Price_Premium_%
PC4 (corr with days on market +0.54): driven by Price_Premium_%, Listing_Photos, List_Price
```

Insights for Maggie

The components that most separate unsold properties from sold ones point to three causes, in order of how strongly they are linked to days on market:

1. **Overpricing.** Unsold properties are listed well above their market value (a higher `Price_Premium_%`). This is the strongest single driver of days on market, so price reductions towards market value are the first fix.
2. **Condition and age.** Older properties with a low `Condition_Score` stay on the market longer, so minor renovation or staging before listing should help.
3. **Location.** The location component (high `Crime_Rate`, low `School_Rating`, long `Distance_to_City_km`) is clearly worse for unsold properties. These properties need pricing that reflects the location, rather than prices copied from befler areas.

Size (area, bedrooms, bathrooms) forms its own component but is barely linked to days on market, so size is **not** the reason properties go unsold. Listings with fewer photos also sell a lifle more slowly, which is an easy marketing fix.

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

The table gives the **loadings** of 9 variables on the first 3 principal components. A loading is the correlation between a variable and a component, so a large absolute loading means that variable strongly defines the component.

```
loadings = pd.read_csv('07-pca-dataset-task-2.csv', index_col='Variable')
loadings
```

Variable	PC1	PC2	PC3
Climate	0.190	0.017	0.207
Housing	0.544	0.020	0.204
Health	0.782	-0.605	0.144
Crime	0.365	0.294	0.585
Transportation	0.585	0.085	0.234
Education	0.394	-0.273	0.027
Arts	0.985	0.126	-0.111
Recreation	0.520	0.402	0.519
Economy	0.142	0.150	0.239

Variance explained by each component = sum of squared loadings in that column. The component with the largest value is the first (most important) principal component.

```
variance = (loadings ** 2).sum()
print("Sum of squared loadings (variance explained by each PC)")
print("-----------------------------------------------------")
print(variance.round(3))
print(f"\nShare of the variance captured by these 3 PCs: {(variance / variance.sum() * 100).r
```

```
Sum of squared loadings (variance explained by each PC)
---------------------------
PC1    2.835
PC2    0.735
PC3    0.842
dtype: float64

Share of the variance captured by these 3 PCs: {'PC1': 64.3, 'PC2': 16.7, 'PC3': 19.1}
```

Significant variables per component: variables with |loading| > 0.5 (these are the bolded values in the exercise table).

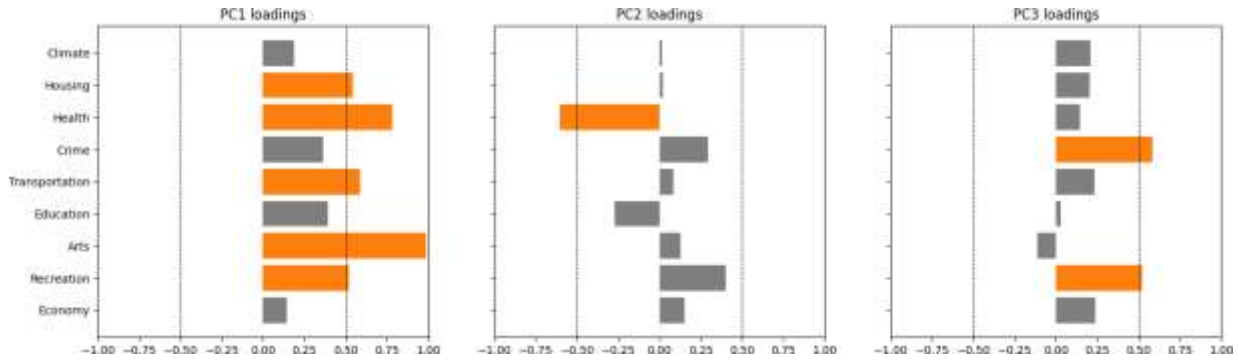
```
threshold = 0.5
loadings.style.apply(lambda col: ['font-weight: bold; background-color: #ffe08a' if abs(v) >
```

Variable	PC1	PC2	PC3
Climate	0.190000	0.017000	0.207000
Housing	**0.544000**	0.020000	0.204000
Health	**0.782000**	**-0.605000**	0.144000
Crime	0.365000	0.294000	**0.585000**
Transportation	**0.585000**	0.085000	0.234000
Education	0.394000	-0.273000	0.027000
Arts	**0.985000**	0.126000	-0.111000
Recreation	**0.520000**	0.402000	**0.519000**
Economy	0.142000	0.150000	0.239000

```
for pc in loadings.columns:
    strong = loadings[pc][loadings[pc].abs() > threshold].sort_values(key=abs, ascending=Fals
    top = loadings[pc].abs().idxmax()
    print(f"{pc}: strongest variable = {top} ({loadings.loc[top, pc]:+.3f})")
    print(f"      significant (|loading| > {threshold}): {', '.join(f'{v} ({l:+.3f})' for v, l
```

```
PC1: strongest variable = Arts (+0.985)
     significant (|loading| > 0.5): Arts (+0.985), Health (+0.782), Transportation (+0.585), H
PC2: strongest variable = Health (-0.605)
     significant (|loading| > 0.5): Health (-0.605)
PC3: strongest variable = Crime (+0.585)
     significant (|loading| > 0.5): Crime (+0.585), Recreation (+0.519)
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5), sharey=True)
for ax, pc in zip(axes, loadings.columns):
    vals = loadings[pc]
    ax.barh(vals.index, vals, color=['tab:orange' if abs(v) > threshold else 'tab:grey' for v
    ax.axvline(threshold, color='k', linestyle='--', linewidth=0.8)
    ax.axvline(-threshold, color='k', linestyle='--', linewidth=0.8)
    ax.set_title(f'{pc} loadings'); ax.set_xlim(-1, 1)
axes[0].invert_yaxis()
plt.show()
```



Communality = sum of squared loadings across the 3 components, i.e. how much of each variable's variance these components explain.

```
communality = (loadings ** 2).sum(axis=1).sort_values(ascending=False)
print(communality.round(3))
```

```
Variable
Arts          0.998
Health        0.998
Recreation    0.701
```

```
Crime           0.562
Transportation  0.404
Housing         0.338
Education       0.230
Economy         0.100
Climate         0.079
dtype: float64
```

```
pc1 = loadings['PC1'].sort_values(ascending=False)
print("Answer")
print("----------")
print(f"PC1 explains the most variance ({variance['PC1']:.3f} vs {variance['PC2']:.3f} and {v
print(f"The variable with the highest loading on PC1 is {pc1.index[0]} ({pc1.iloc[0]:.3f}), f
```

```
Answer
----------
PC1 explains the most variance (2.835 vs 0.735 and 0.842), so it is the principal component.
The variable with the highest loading on PC1 is Arts (0.985), followed by Health (0.782).
```

Conclusion

- **PC1 is the principal component.** It explains far more variance than PC2 or PC3, and **Arts (0.985)** is the variable that dominates it, followed by Health (0.782), Transportation (0.585), Housing (0.544) and Recreation (0.520). All nine loadings on PC1 are positive, so PC1 measures the overall urban amenities and quality of life of a place, driven mainly by Arts and Health.
- **PC2** is defined mainly by **Health (−0.605)**, contrasted against Recreation (0.402). A high PC2 score means befler recreation but poorer health care.
- **PC3** is defined by **Crime (0.585)** and **Recreation (0.519)**.
- **Arts** and **Health** have the highest communality (0.998 each, so they are almost fully explained by these 3 PCs), while **Climate** and **Economy** load weakly on all three.

Name: GUNTI RAHUL

Roll Number: CH.SC.U4CSE24211

```
import numpy as np
import pandas as pd
from collections import Counter
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Problem: the fruit basket is the data. *n* random samples are taken from the basket and an individual decision tree is built for each sample. Each tree gives its own output for a new fruit (the *instance*), and **majority voting** over all trees gives the final class. Which fruit is likely to be taken most often?

Dataset: a synthetic fruit basket with the weight, diameter, length, sweetness and colour of each fruit.

```
basket = pd.read_csv('08-rf-dataset-task-1.csv')
```

```
basket.head()
```

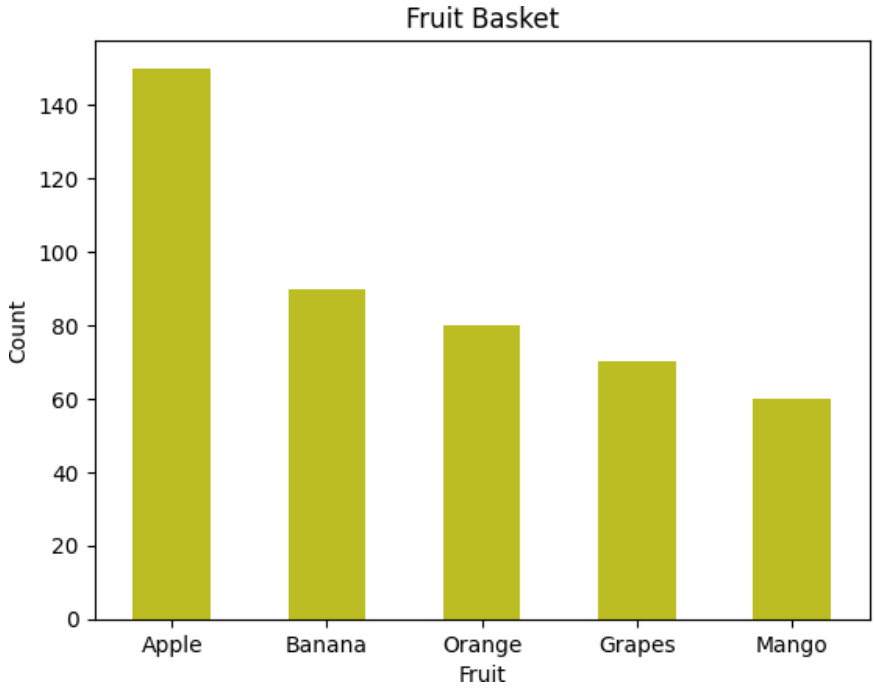
	Weight_g	Diameter_cm	Length_cm	Sweetness	Colour	Fruit
0	2.2	2.1	2.4	7.1	Green	Grapes
1	4.8	1.9	2.8	6.9	Green	Grapes
2	160.7	7.8	8.6	5.7	Green	Apple
3	224.2	8.5	8.7	6.8	Red	Apple
4	2.6	1.9	2.2	7.4	Purple	Grapes

```
basket.shape
```

```
(450, 6)
```

```
counts = basket['Fruit'].value_counts()
print("Fruits in the basket")
print("--------------------------------------------")
print(counts)
counts.plot.bar(color='tab:olive', title='Fruit Basket', ylabel='Count', rot=0)
plt.show()
```

```
Fruits in the basket
--------------------------------------------------
Fruit
Apple     150
Banana     90
Orange     80
Grapes     70
Mango      60
Name: count, dtype: int64
```



Colour is a text feature, so it is one-hot encoded.

```
X = pd.get_dummies(basket.drop(columns='Fruit'), columns=['Colour'], dtype=int)
y = basket['Fruit']
X.head()
```

	Weight_g	Diameter_cm	Length_cm	Sweetness	Colour_Green	Colour_Orange	Colour_Purple	Colo
0	2.2	2.1	2.4	7.1	1	0	0	
1	4.8	1.9	2.8	6.9	1	0	0	
2	160.7	7.8	8.6	5.7	1	0	0	
3	224.2	8.5	8.7	6.8	0	0	0	
4	2.6	1.9	2.2	7.4	0	0	1	

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, str
```

* Building the forest by hand (as in the figure)

Each tree gets a **bootstrap sample**: fruits drawn at random *with replacement* from the basket, the same size as the training set. Each tree also considers only a random subset of features at each split (`max_features='sqrt'`), so the trees differ from each other.

```
n_trees = 25
rng = np.random.default_rng(42)
trees, samples = [], []
for i in range(n_trees):
    idx = rng.integers(0, len(X_train), len(X_train))          # sample with replacement
    tree = DecisionTreeClassifier(max_features='sqrt', random_state=i)
```

```
        tree.fit(X_train.iloc[idx], y_train.iloc[idx])
        trees.append(tree)
        samples.append(y_train.iloc[idx])
print(f"Built {len(trees)} decision trees, each on {len(X_train)} fruits sampled with replace
```

```
Built 25 decision trees, each on 360 fruits sampled with replacement
```

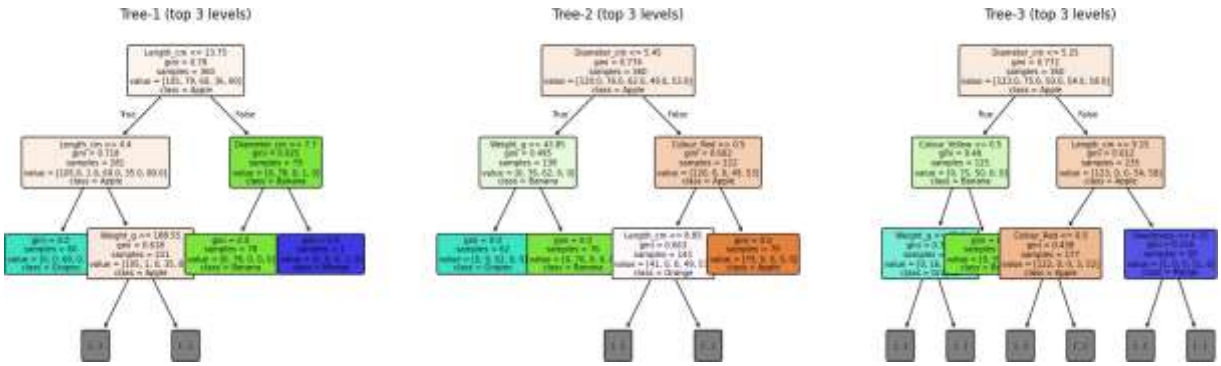
```
sample_counts = pd.DataFrame([s.value_counts() for s in samples]).fillna(0).astype(int)
sample_counts.index = [f'Tree-{i + 1}' for i in range(n_trees)]
print("How many of each fruit went into each tree's sample (first 5 trees)")
print("-------------------------------------------------")
print(sample_counts.head())
print("\nAverage per sample:")
print(sample_counts.mean().round(1).sort_values(ascending=False))
```

```
How many of each fruit went into each tree's sample (first 5 trees)
-------------------------------------------------
```

Fruit	Apple	Orange	Banana	Grapes	Mango
Tree-1	105	80	79	60	36
Tree-2	120	53	76	62	49
Tree-3	123	58	75	50	54
Tree-4	127	67	67	53	46
Tree-5	135	65	56	50	54

```
Average per sample:
Fruit
Apple     121.4
Banana     70.9
Orange     63.9
Grapes     54.2
Mango      49.7
dtype: float64
```

```
fig, axes = plt.subplots(1, 3, figsize=(20, 6))
for ax, i in zip(axes, range(3)):
    plot_tree(trees[i], feature_names=X.columns, class_names=trees[i].classes_, max_depth=2,
              filled=True, rounded=True, fontsize=7, ax=ax)
    ax.set_title(f'Tree-{i + 1} (top 3 levels)')
plt.show()
```



* A new fruit (the instance)

A yellow, round fruit of about 170 g. Yellow apples and oranges look alike, so the trees may not all agree. Every tree classifies it independently.

```
instance = pd.DataFrame([{'Weight_g': 170, 'Diameter_cm': 7.9, 'Length_cm': 8.3, 'Sweetness':
instance_X = pd.get_dummies(instance, columns=['Colour'], dtype=int).reindex(columns=X.column
instance
```

	Weight_g	Diameter_cm	Length_cm	Sweetness	Colour
0	170	7.9	8.3	6.5	Yellow

```
votes = [tree.predict(instance_X)[0] for tree in trees]
for i, v in enumerate(votes, 1):
    print(f"Tree-{i:<3} -> {v}")
```

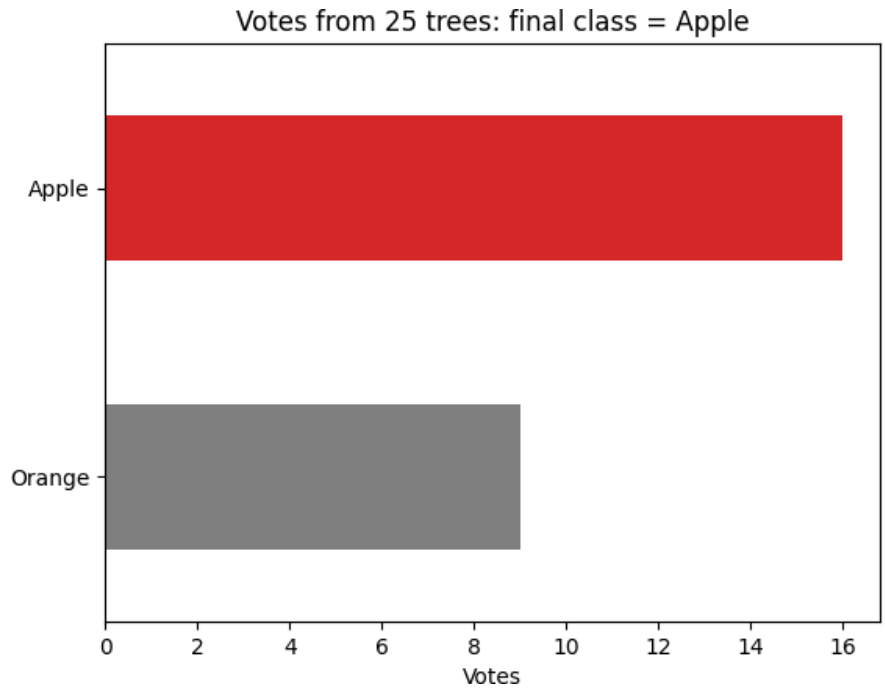
```
Tree-1   -> Orange
Tree-2   -> Apple
Tree-3   -> Orange
Tree-4   -> Apple
Tree-5   -> Apple
Tree-6   -> Orange
Tree-7   -> Orange
Tree-8   -> Apple
Tree-9   -> Apple
Tree-10  -> Orange
Tree-11  -> Orange
Tree-12  -> Orange
Tree-13  -> Apple
Tree-14  -> Orange
Tree-15  -> Apple
Tree-16  -> Orange
Tree-17  -> Apple
Tree-18  -> Apple
Tree-19  -> Apple
Tree-20  -> Apple
Tree-21  -> Apple
Tree-22  -> Apple
Tree-23  -> Apple
Tree-24  -> Apple
Tree-25  -> Apple
```

```
tally = Counter(votes)
final_class = tally.most_common(1)[0][0]
print("Majority Voting")
print("----------------------------------------------")
for fruit, n in tally.most_common():
    print(f"{fruit:<8} {n} votes")
print(f"\nFinal class: {final_class}")
```

```
Majority Voting
----------------------------
Apple    16 votes
Orange   9 votes

Final class: Apple
```

```
pd.Series(tally).sort_values().plot.barh(color=['tab:red' if f == final_class else 'tab:grey'
plt.xlabel('Votes'); plt.title(f'Votes from {n_trees} trees: final class = {final_class}')
plt.show()
```



* The forest on the whole test set

The same majority vote applied to every fruit in the test set, compared with a single decision tree.

```
all_votes = np.array([tree.predict(X_test) for tree in trees])
forest_pred = np.array([Counter(col).most_common(1)[0][0] for col in all_votes.T])
single_tree = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)

print(f"Single decision tree accuracy: {accuracy_score(y_test, single_tree.predict(X_test)):.
print(f"Average accuracy of the individual trees: {np.mean([accuracy_score(y_test, p) for p i
print(f"Hand-built forest (majority vote) accuracy: {accuracy_score(y_test, forest_pred):.4f}
```

```
Single decision tree accuracy: 0.9556
Average accuracy of the individual trees: 0.9218
Hand-built forest (majority vote) accuracy: 0.9444
```

Each individual tree is trained on only part of the data and a random subset of features, so on its own it is weaker than a full decision tree. Majority voting lifts the forest above the average individual tree. On this dataset the fruits are easy to separate, so a single full tree already does about as well; the benefit of a forest shows most on noisier data, where single trees overfit.

* scikit-learn's RandomForestClassifier

The same idea (bootstrap samples + random feature subsets + majority voting) in one class.

```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
▾ RandomForestClassifier ⓘ ?
▸ Parameters
```

```
predictions = model.predict(X_test)
```

```
print("Actual vs Predicted Fruit")
print("--------------------------------------------------")
comparison_df = pd.DataFrame({
```

```
    'Actual': y_test.values,
    'Predicted': predictions
})
print(comparison_df)
```

```
Actual vs Predicted Fruit
----------------------------------------
    Actual Predicted
0    Mango     Mango
1    Apple     Apple
2   Grapes    Grapes
3   Orange    Orange
4   Banana    Banana
..     ...       ...
85   Apple     Apple
86  Grapes    Grapes
87  Grapes    Grapes
88  Grapes    Grapes
89  Banana    Banana

[90 rows x 2 columns]
```

```
acc = accuracy_score(y_test, predictions)

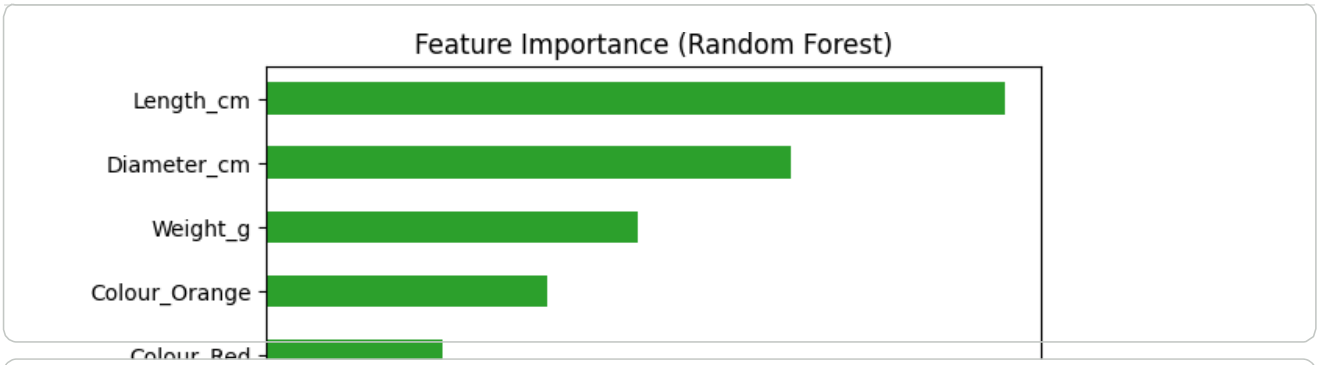
print("Classification Evaluation Metrics")
print("----------------------------------------------------")
print(f"Accuracy: {acc:.4f}")
print("Confusion Matrix:")
print(confusion_matrix(y_test, predictions, labels=model.classes_))
print("Classification Report:")
print(classification_report(y_test, predictions))
```

```
Classification Evaluation Metrics
----------------------------
Accuracy: 0.9667
Confusion Matrix:
[[29  0  0  0  1]
 [ 0 18  0  0  0]
 [ 0  0 14  0  0]
 [ 0  0  0 12  0]
 [ 2  0  0  0 14]]
Classification Report:
              precision    recall  f1-score   support

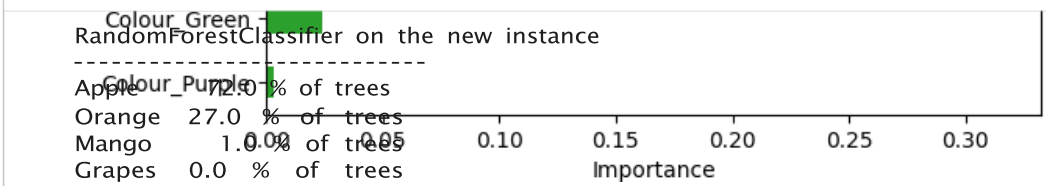
       Apple       0.94      0.97      0.95        30
      Banana       1.00      1.00      1.00        18
      Grapes       1.00      1.00      1.00        14
       Mango       1.00      1.00      1.00        12
      Orange       0.93      0.88      0.90        16

    accuracy                           0.97        90
   macro avg       0.97      0.97      0.97        90
weighted avg       0.97      0.97      0.97        90
```

```
importance = pd.Series(model.feature_importances_, index=X.columns).sort_values()
importance.plot.barh(color='tab:green')
plt.xlabel('Importance'); plt.title('Feature Importance (Random Forest)')
plt.show()
```




```
proba = pd.Series(model.predict_proba(instance_X)[0], index=model.classes_).sort_values(ascen
print("RandomForestClassifier on the new instance")
print("---------------------------------------------- ")
print((proba * 100).round(1).astype(str) + ' % of trees')
print(f"\nFinal class: {model.predict(instance_X)[0]}")
```




```
RandomForestClassifier on the new instance
----------------------------
Apple   72.0 % of trees
Orange  27.0 % of trees
Mango    1.0 % of trees
Grapes   0.0 % of trees
Banana   0.0 % of trees
dtype: object

Final class: Apple
```